YOLO environment construction

YOLO environment construction

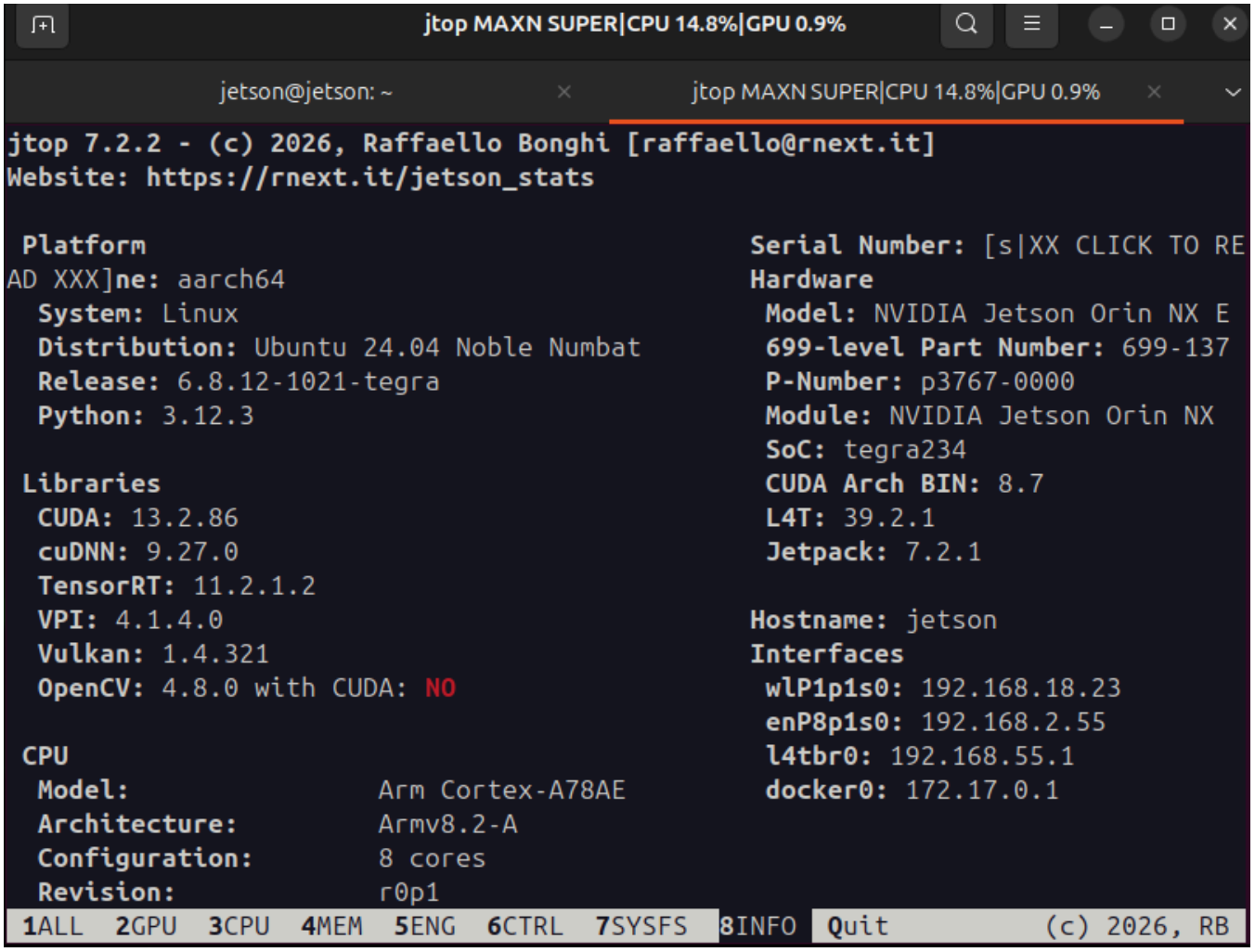
- 1. System information
- 2. Preliminary preparation
- 3. Install Ultralytics
- 4. Configure the GPU acceleration
  - 4.1. PyTorch GPU environment
  - 4.2. Torchvision is missing the nvJPEG runtime library
- 5. Verify the installation
- Common errors
  - Cannot uninstall sympy
    - Symptom
    - Solution
  - The CSI camera cannot be called
    - Verify the environment
- References

> **A note on the pip installation:** Under PEP 668, Ubuntu 24.04 marks the system Python as "externally managed", so installing with pip directly may report `externally-managed-environment`. This tutorial installs into the home directory of the current user with `--user --break-system-packages`. A virtual environment usually does not need these parameters.

1. System information

Applicable environment: Jetson Orin NX, JetPack 7.2.1 / L4T R39.2.1, Ubuntu 24.04, Python 3.12.

This tutorial uses the bundled Ultralytics 8.3.55 source code and reuses the PyTorch 2.8.0, the Torchvision 0.23.0 and the system OpenCV that the previous tutorials have already configured. The default CUDA toolchain of the system is 13.2, while the current PyTorch build uses CUDA 12.9; the two do not have to be the same version number.



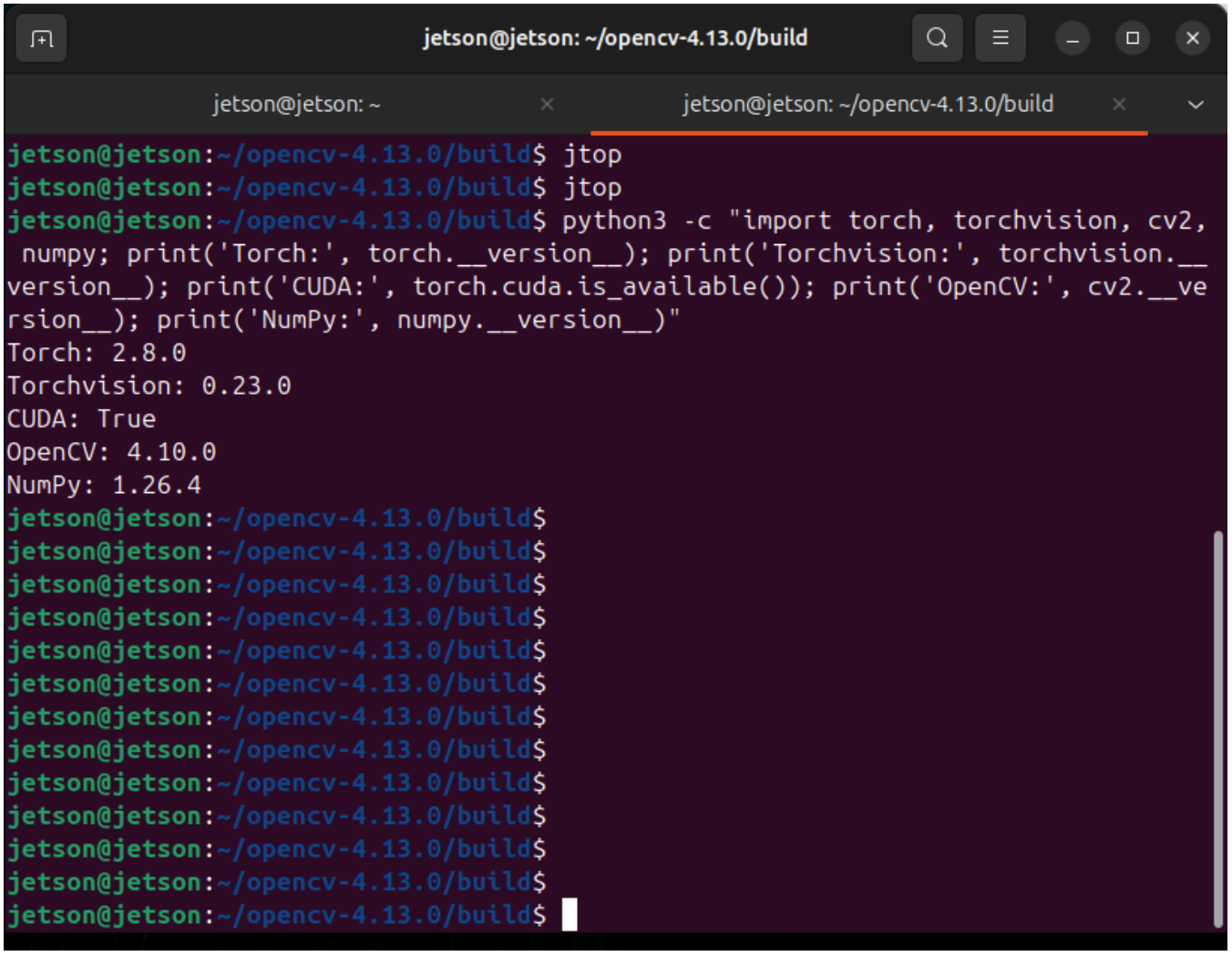
2. Preliminary preparation

Note: if the factory image already contains the environment of this tutorial, you only have to run the verification of section 5. When you build the system yourself, first complete the PyTorch and the Torchvision environment construction of the previous tutorials and then run the installation steps. The commands below are run in the terminal of the Jetson.

Check the frameworks that are already there:

```
python3 -c "import torch, torchvision, cv2, numpy; print('Torch:', torch.__version__);
print('Torchvision:', torchvision.__version__); print('CUDA:', torch.cuda.is_available());
print('OpenCV:', cv2.__version__); print('NumPy:', numpy.__version__)"
```

The expectation is that the PyTorch is `2.8.0`, that the Torchvision is `0.23.0`, that the CUDA is `True` and that the NumPy is `1.26.4`. The OpenCV uses the version that the current system provides; the GStreamer support that the CSI camera needs is checked in the common errors. If the import fails or the CUDA is `False`, first sort out the framework environment instead of continuing to install the YOLO dependencies to try to fix it.



Install pip:

```
sudo apt update
sudo apt install -y python3-pip
```

This tutorial reuses the basic dependencies that are already there, such as `matplotlib`, `pillow`, `pyyaml`, `requests`, `scipy`, `psutil` and `pandas`. You can check them first:

```
python3 -c "import matplotlib, PIL, yaml, requests, scipy, psutil, pandas; print('base dependencies import OK')"
```

If a module is reported as missing, that dependency has to be filled in first. The `--no-deps` installation of this tutorial does not install the missing dependencies automatically, and it does not apply to a pure system whose basic frameworks are not ready yet.

3. Install Ultralytics

Copy the `ultralytics.zip` of the bundled material to the Jetson and extract it into the home directory of the current user. After the extraction, confirm that `~/ultralytics/pyproject.toml` exists and that the directory also contains the `ultralytics/` source code subdirectory.

If `~/ultralytics` already contains source code or training data, do not overwrite it directly; first check the current version and the purpose of the directory. The version of the bundled source code is `8.3.55`.

Enter the source code directory, fill in the basic dependencies and install:

```
cd ~/ultralytics

python3 -m pip install --user --break-system-packages --no-deps \
    tqdm py-cpuinfo seaborn ultralytics-thop

python3 -m pip install --user --break-system-packages --no-deps -e .

export PATH="$HOME/.local/bin:$PATH"
```

`--no-deps` is used to prevent pip from automatically replacing the PyTorch, the Torchvision and the system OpenCV that have already been verified; `-e .` means that the local source code is used directly, so the later modifications of the source code take effect.

The `export` only takes effect in the current terminal. If a new terminal reports that `yolo` cannot be found, you can run the command again, or you can use `~/.local/bin/yolo` directly.

Do not use `sudo pip3 install ultralytics[export]` instead of the steps above. The `[export]` additional dependency of the bundled 8.3.55 requires the old NumPy 1.23.5 to be installed on ARM64 and it brings in other export frameworks, which does not fit the current Python 3.12 environment. The ordinary user does not have to restart the system after the installation is complete.

4. Configure the GPU acceleration

4.1. PyTorch GPU environment

The PyTorch 2.8.0 and the Torchvision 0.23.0 have already been installed and verified above.

```
python3 -c "import torch; print('Torch:', torch.__version__); print('CUDA used in the
build:', torch.version.cuda); print('CUDA available:', torch.cuda.is_available());
print('GPU:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else
'unavailable')"
```

Python 3.12 cannot install the PyTorch, the Torchvision or the ONNX Runtime wheels whose file name contains `cp310`. Whether the cuSPARSELt is needed has to be based on the actual PyTorch build dependencies and on the current system software source; it is not a uniform installation step of this section.



4.2. Torchvision is missing the nvJPEG runtime library

If importing the Torchvision reports `libnvjpeg.so.12: cannot open shared object file`, the adaptation method for the current CUDA 12.9 build is:

```
sudo apt-get install -y libnvjpeg-12-9
sudo ldconfig

ldconfig -p | grep libnvjpeg
python3 -c "import torchvision; print(torchvision.__version__)"
```

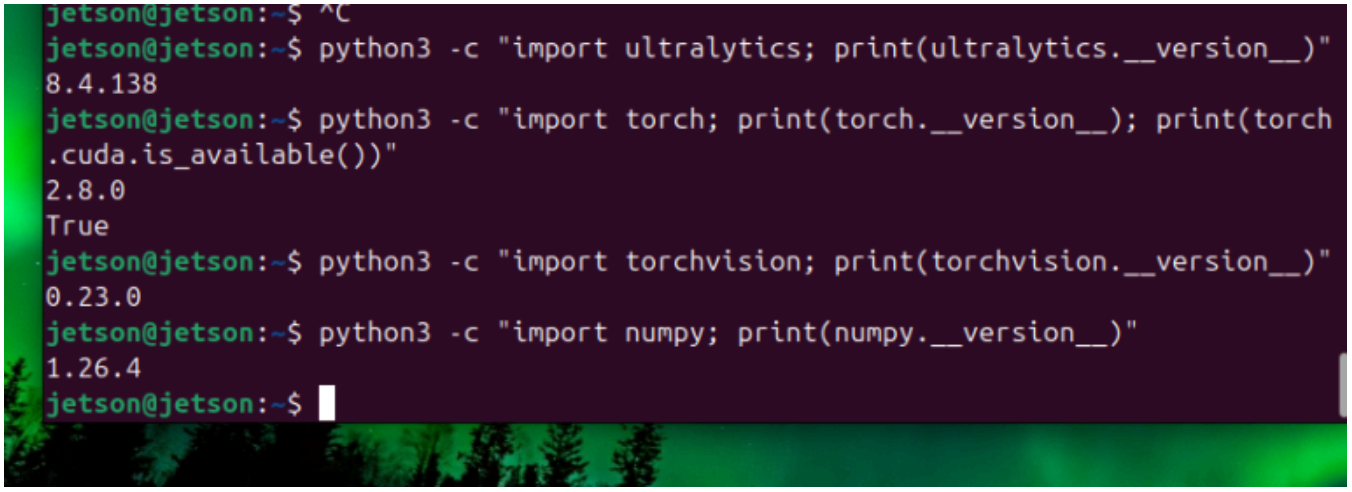
When there is no such error you do not have to install it again. This fills in the nvJPEG runtime library of the CUDA 12.9 and it does not require switching the default CUDA toolchain of the system.

5. Verify the installation

> **Verify Torch, Torchvision and NumPy**

```
python3 -c "import ultralytics; print(ultralytics.__version__)"
python3 -c "import torch; print(torch.__version__); print(torch.cuda.is_available())"
python3 -c "import torchvision; print(torchvision.__version__)"
python3 -c "import numpy; print(numpy.__version__)"
```

The expected result is the same as in the picture below:



> **Verify the actual GPU inference**

On the premise that the bundled source code contains the `yolo11n.pt` weights and the `bus.jpg` picture, run:

```
cd ~/ultralytics
ls -l yolo11n.pt ultralytics/assets/bus.jpg
yolo predict model=yolo11n.pt source=ultralytics/assets/bus.jpg device=0
```

When the weights are missing, fill them in from the bundled material first. The inference log should show `CUDA:0 (Orin)` and it should output the detection results normally. The results are saved in `runs/detect/predict` by default and a repeated run is numbered automatically. This verification generates a result picture, and you can delete the result directory of this run when you are done with it.

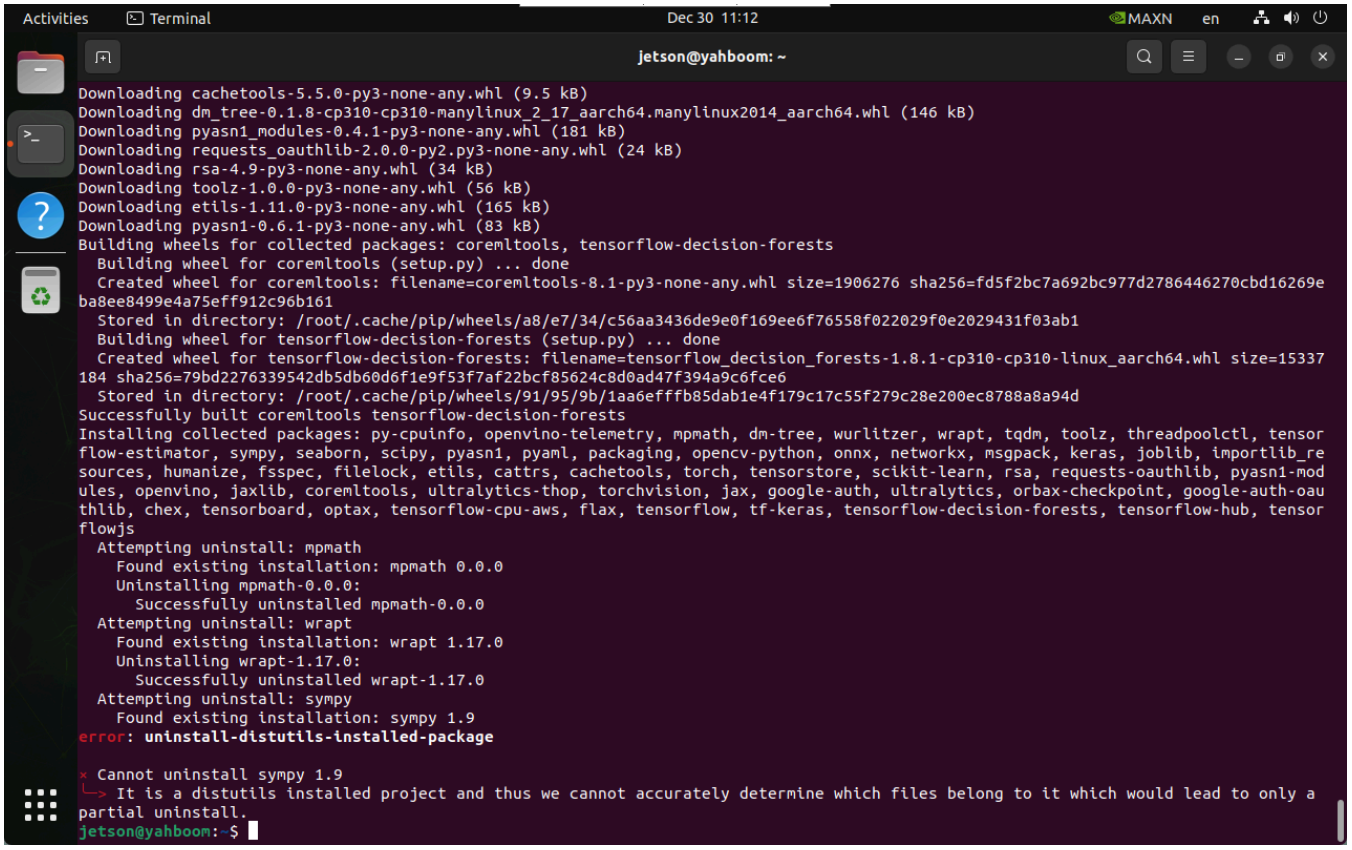
Seeing only `torch.cuda.is_available() = True` does not mean that the actual YOLO inference has passed, and it does not prove that the ONNX Runtime uses the GPU.

Common errors

Cannot uninstall sympy

Symptom

While installing or upgrading the framework it reports that sympy cannot be uninstalled.



The picture above is an example of the error of an old environment.

Solution

First confirm whether the command that is running tries to reinstall the PyTorch or another framework. This tutorial uses the user level `--no-deps` installation, it reuses the existing dependencies and it does not require uninstalling the system `python3-sympy`.

```
python3 -m pip show sympy
python3 -c "import sympy; print(sympy.__version__); print(sympy.__file__)"
```

If there really is a version conflict, handle it after you have confirmed the required version and the installation location according to the error; do not uninstall the system sympy directly, as that would also remove the other APT packages.

The CSI camera cannot be called

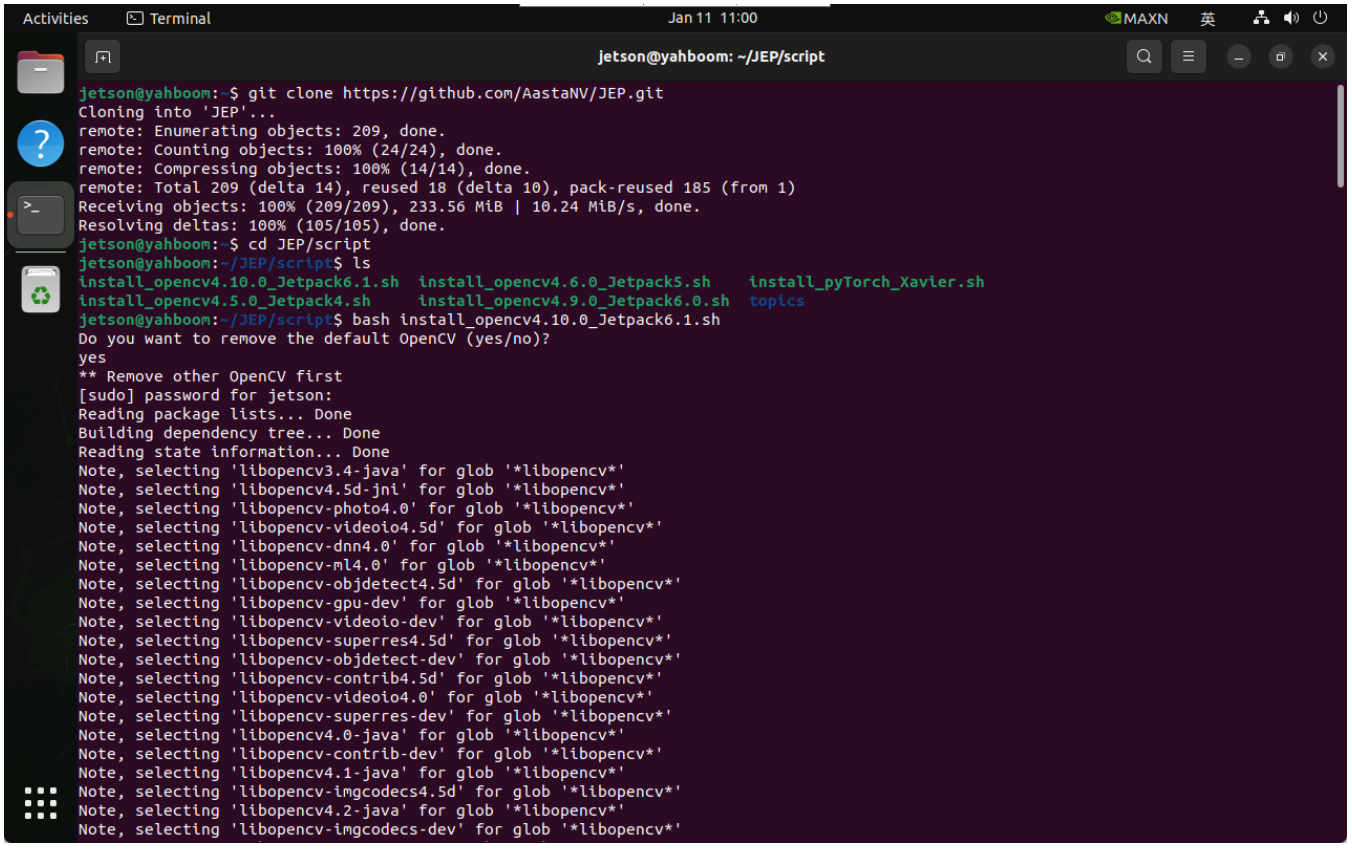
First check whether the OpenCV supports the GStreamer, whether the camera device exists and whether it is occupied by another program:

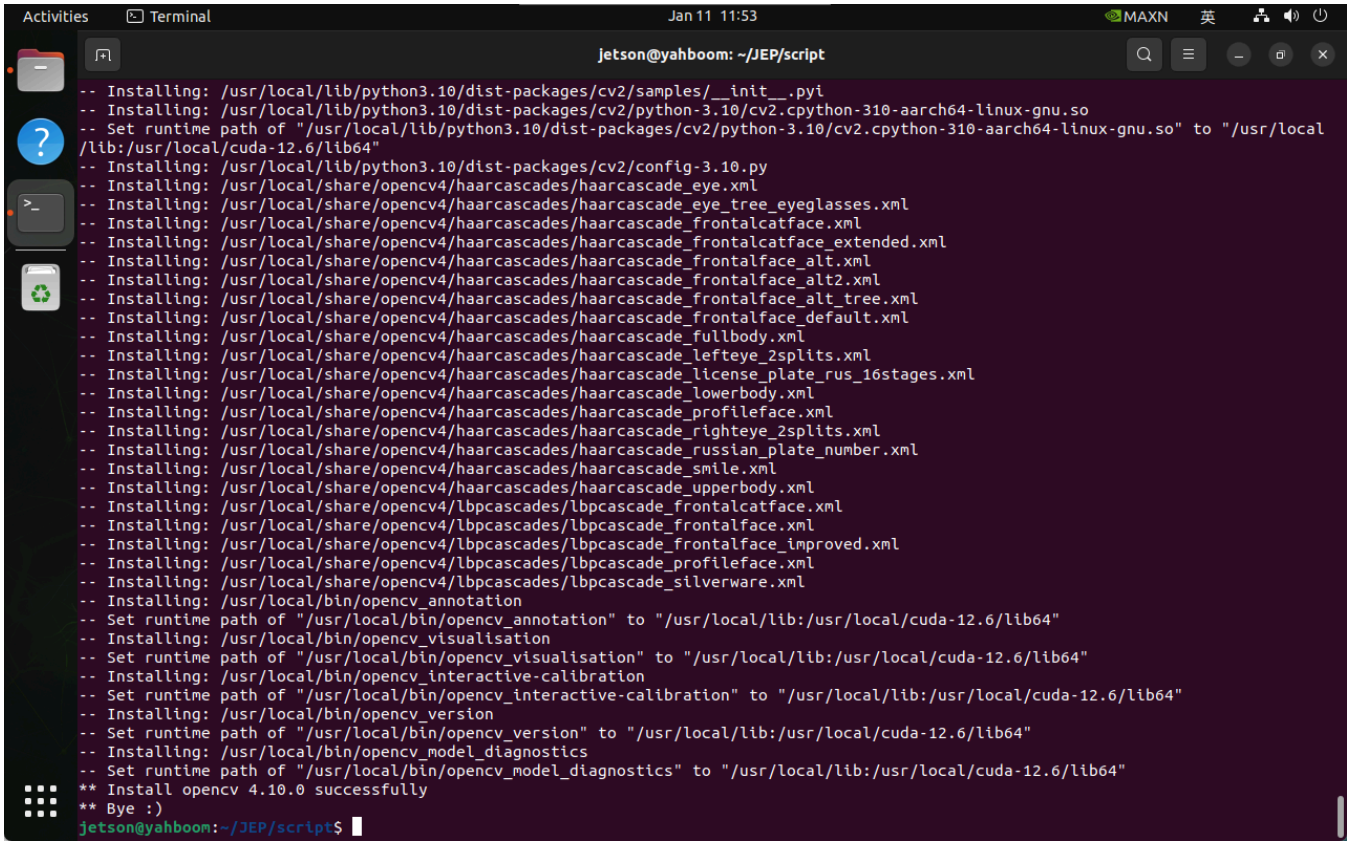
```
python3 -c "import cv2; print(cv2.__version__); print(cv2.__file__);
print(cv2.getBuildInformation())" | grep -E 'GStreamer|GUI|site-packages|dist-packages'
ls -l /dev/video*
sudo fuser -v /dev/video0
systemctl status nvargus-daemon --no-pager
```

`/dev/video0` is only a common example; the actual number has to be chosen according to the result of the device check. When it shows `GStreamer: YES`, there is no need to recompile the OpenCV just for YOLO.

Programs such as the ROS2 `camera_csi` may be occupying the CSI camera. After you confirm the occupying program, first exit the corresponding camera application normally and then test the YOLO; do not directly end an unknown process. When there is no graphical display environment over SSH, the window cannot be shown and that alone does not mean that the camera driver is faulty.

The pictures below are the reference of the compilation process of the old OpenCV and of the environment verification; they are not the installation steps of the current JetPack 7.2.1. Do not run `install_opencv4.10.0_Jetpack6.1.sh` in this environment; only after you have confirmed that the GStreamer function is missing do you have to prepare a separate compilation plan that matches the current JetPack, CUDA and Python.





Verify the environment

```
python3 -c "import cv2; print(cv2.getBuildInformation())" | grep GStreamer
python3 -c "import ultralytics; print(ultralytics.__version__)"
python3 -c "import torch; print(torch.__version__); print(torch.cuda.is_available())"
python3 -c "import torchvision; print(torchvision.__version__)"
python3 -c "import numpy; print(numpy.__version__)"
jtop
```

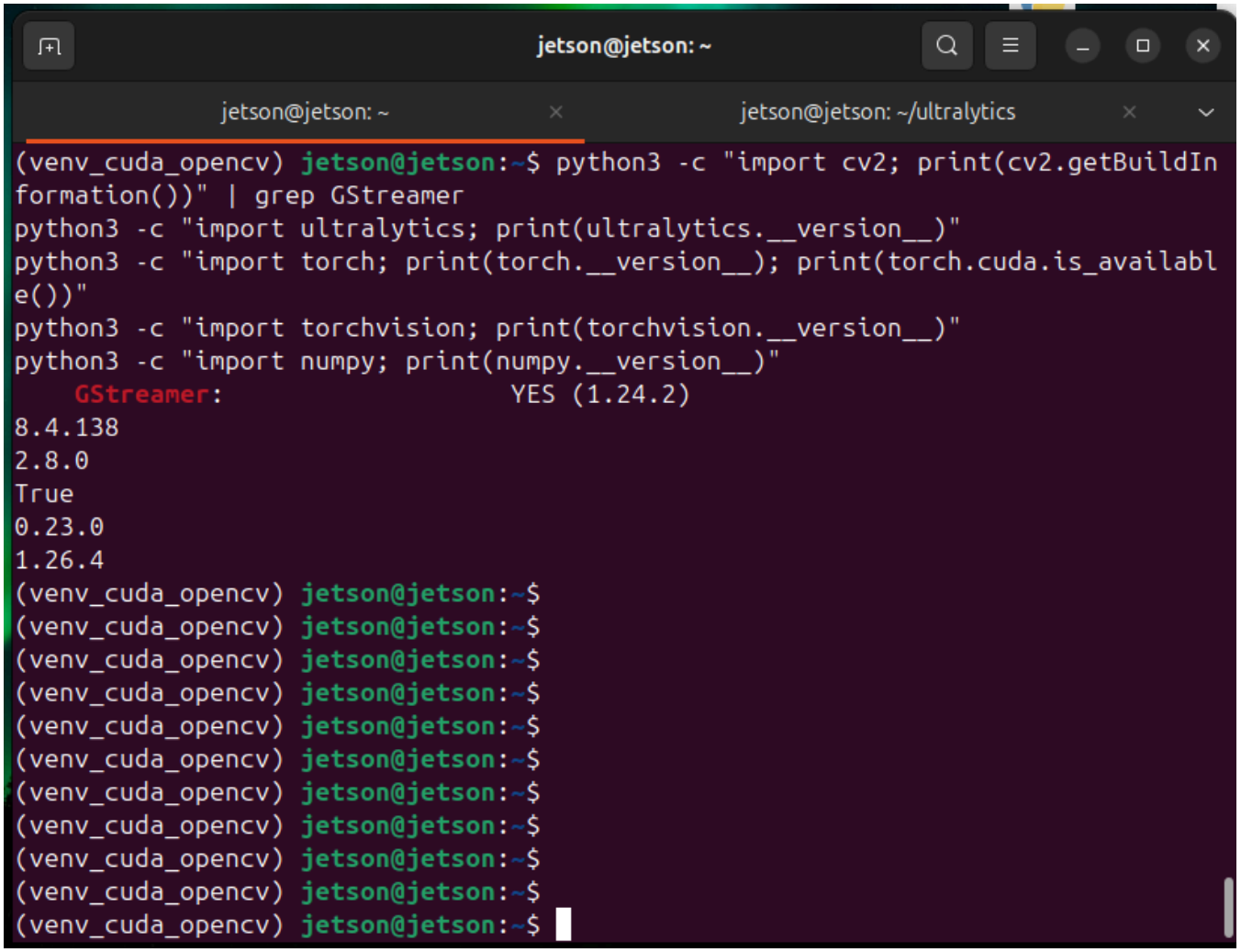
The picture below is the reference of the old environment verification; for the current version the expected values of section 5 and the actual query result are the reference,

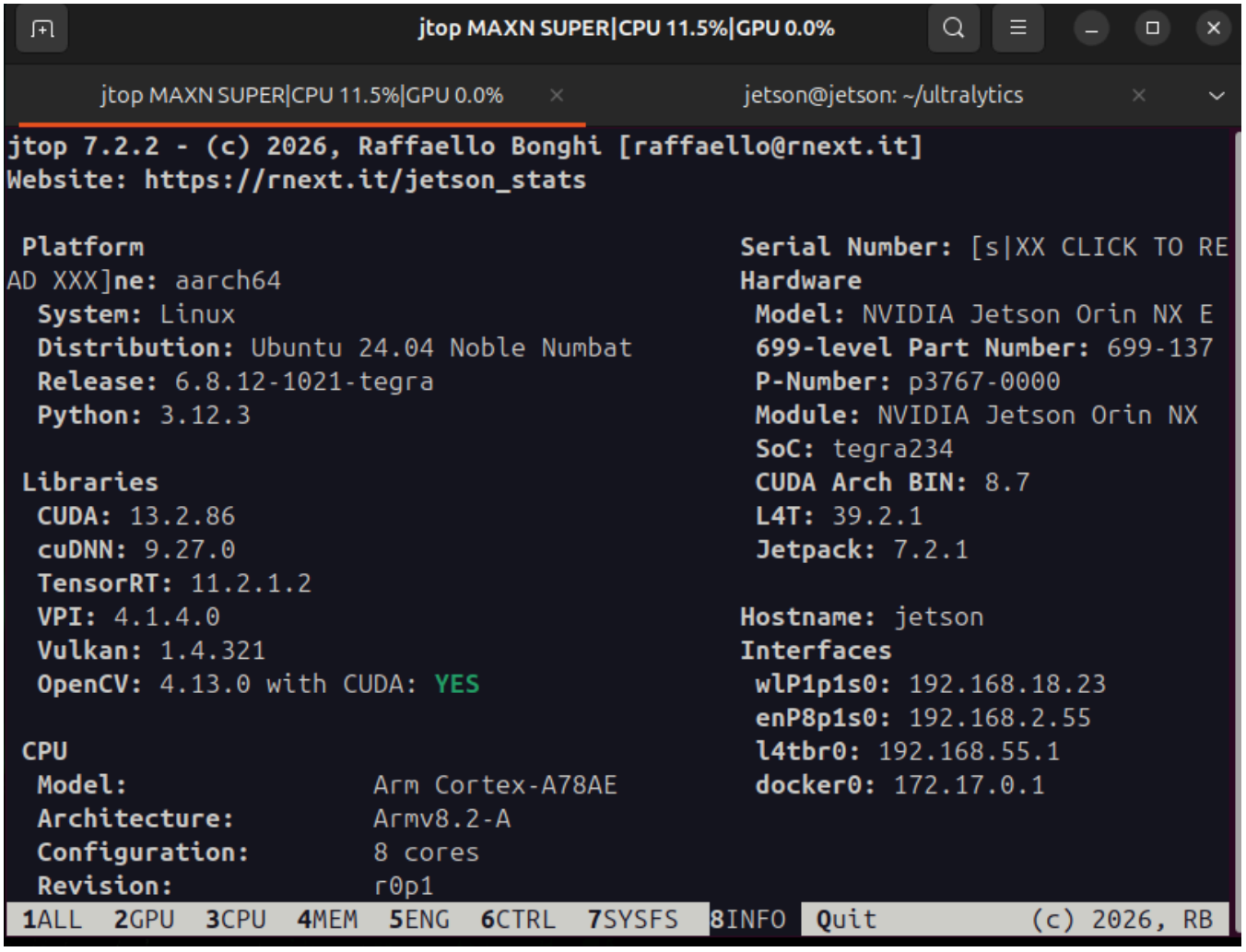
The opencv that is installed in the default environment is the 4.8 version, and this environment does not have a cuda environment that can be paired with the 13.2 version, so the opencv that is used runs with the CPU acceleration. If you want to run the related cases with the GPU version of the opencv, you can enter the virtual environment that we have set up.

```
source ~/venv_cuda_opencv/bin/activate
```

If you find that the opencv of this environment reports an error when it runs our cases, you can exit the virtual environment and switch back to the initial environment to operate.

```
deactivate
```


jetson@jetson: ~
jetson@jetson: ~
jetson@jetson: ~/ultralytics
(venv_cuda_opencv) jetson@jetson:~$ python3 -c "import cv2; print(cv2.getBuildInformation())" | grep GStreamer
python3 -c "import ultralytics; print(ultralytics.__version__)"
python3 -c "import torch; print(torch.__version__); print(torch.cuda.is_available())"
python3 -c "import torchvision; print(torchvision.__version__)"
python3 -c "import numpy; print(numpy.__version__)"
GStreamer: YES (1.24.2)
8.4.138
2.8.0
True
0.23.0
1.26.4
(venv_cuda_opencv) jetson@jetson:~$
(venv_cuda_opencv) jetson@jetson:~$
(venv_cuda_opencv) jetson@jetson:~$
(venv_cuda_opencv) jetson@jetson:~$
(venv_cuda_opencv) jetson@jetson:~$
(venv_cuda_opencv) jetson@jetson:~$
(venv_cuda_opencv) jetson@jetson:~$
(venv_cuda_opencv) jetson@jetson:~$
(venv_cuda_opencv) jetson@jetson:~$
(venv_cuda_opencv) jetson@jetson:~$



References

The Ultralytics Jetson deployment guide

AastaNV/JEP (the reference of the old compilation script)